

15 — Fonctionnement détaillé : La pagination

Explication du fonctionnement réel (tel que codé), pour répondre à l'oral « comment marche la pagination ? ».

En une phrase

On charge **toutes** les données d'un coup depuis l'API, on les garde dans une variable, puis on n'**affiche que 10 ou 24 éléments à la fois** en découpant le tableau. Des boutons « ← Précédent » et « Suivant → » changent la tranche affichée. **Tout se passe côté navigateur (frontend), rien de spécial côté serveur.**

C'est ce qu'on appelle une **pagination côté client**.

Où est-elle utilisée ?

Page	Fichier	Éléments par page
Marketplace des annonces	Frontend/script/annonces/annonceAll.js	24 (ANN_PAR_PAGE)
Liste des utilisateurs (dashboard admin)	Frontend/script/admin/user.js	10 (USERS_PAR_PAGE)

Les deux marchent **exactement pareil**. On explique ci-dessous avec les annonces.

Les 3 ingrédients

En haut du fichier, 3 variables toutes simples :

```
let annoncesAffichees = []; // toutes les annonces reçues de l'API
let pageAnnonce = 1;       // la page où on se trouve
const ANN_PAR_PAGE = 24;   // combien on montre par page
```

- `annoncesAffichees` = la liste complète (par ex. 300 annonces).
 - `pageAnnonce` = le numéro de page actuel (1, 2, 3...).
 - `ANN_PAR_PAGE` = le nombre fixe d'éléments affichés à la fois.
-

Étape 1 — On reçoit les données et on revient page 1

Quand l'API répond, on stocke la liste et on repart de la page 1 :

```
function displayAnnonces(items) {  
  annoncesAffichees = items || [];  
  pageAnnonce = 1;  
  renderPageAnnonces();  
}
```

À noter : cette fonction est aussi appelée quand on **filtre** ou qu'on **cherche**. On remet à la page 1 pour ne pas rester bloqué sur une page vide.

Étape 2 — On découpe la bonne tranche

C'est le cœur de la pagination. On calcule combien de pages il faut, puis on prend **seulement** les 24 éléments de la page courante avec `.slice()` :

```
const nbPages = Math.max(1, Math.ceil(items.length / ANN_PAR_PAGE));  
if (pageAnnonce > nbPages) pageAnnonce = nbPages;  
  
const debut = (pageAnnonce - 1) * ANN_PAR_PAGE;  
const pageItems = items.slice(debut, debut + ANN_PAR_PAGE);
```

Comment lire ce calcul : - `nbPages` = nombre total de pages. `Math.ceil` arrondit **vers le haut** ($300 \div 24 = 12,5 \rightarrow$ **13 pages**). - `debut` = l'index où commence la page. Page 1 $\rightarrow$ 0, page 2 $\rightarrow$ 24, page 3 $\rightarrow$ 48... - `items.slice(debut, debut + 24)` = on récupère juste ces 24 éléments-là.

Ensuite on fait une boucle `forEach` sur `pageItems` pour créer les cartes affichées. **On n'affiche jamais tout, seulement la tranche.**

Étape 3 — La barre de boutons

Si il y a plus d'une page, on ajoute en bas une petite barre « ← Précédent | Page X / Y | Suivant → » :

```
if (nbPages > 1) {  
  // bouton Précédent (désactivé si on est déjà page 1)  
  // texte "Page X / Y"  
  // bouton Suivant (désactivé si on est à la dernière page)  
}
```

Les boutons appellent une seule fonction :

```
function changerPageAnnonce(delta) {  
  pageAnnonce += delta;          // -1 pour reculer, +1 pour avancer  
  if (pageAnnonce < 1) pageAnnonce = 1;  
  renderPageAnnonces();          // on ré-affiche la nouvelle tranche  
  window.scrollTo({ top: 0, behavior: "smooth" }); // on remonte en haut  
}
```

- « Précédent » appelle `changerPageAnnonce(-1)`
- « Suivant » appelle `changerPageAnnonce(1)`

Et on ré-affiche. C'est tout. **Aucun nouvel appel à l'API** : les données sont déjà en mémoire, on change juste la tranche montrée.

Le même principe côté utilisateurs (admin)

Dans `user.js`, mêmes 3 variables (`tousLesUsers`, `pageUsers`, `USERS_PAR_PAGE = 10`), même `.slice()`, même barre de boutons (`changerPageUsers(-1 / +1)`). Seule différence : **10** utilisateurs par page au lieu de 24 annonces.

Questions possibles à l'oral

« **Pourquoi la pagination est côté client et pas côté serveur ?** » Parce que c'est **plus simple à coder** : on récupère toute la liste une fois, puis on découpe en JavaScript. Pas besoin de gérer des `LIMIT / OFFSET` en SQL ni des paramètres de page dans l'API. Pour un projet de cette taille c'est largement suffisant.

« **Et si il y a 50 000 annonces ?** » Là, la pagination côté client montre ses limites (on télécharge tout). La solution serait de passer à une **pagination côté serveur** (l'API renvoie 24 annonces par

page avec `LIMIT 24 OFFSET ...`). C'est justement pour éviter de tout charger qu'on a aussi **nettoyé la base** et ajouté la pagination d'affichage.

« **Pourquoi `Math.ceil` et pas `Math.round` ?** » Parce qu'il faut arrondir **vers le haut** : s'il reste 5 annonces après les pages pleines, il faut quand même **une page de plus** pour les montrer.

`Math.round(12,5)` donnerait 13 par chance, mais `Math.ceil` garantit toujours le bon résultat (12,1 → 13 aussi).

« **Où est stockée la page courante ?** » Dans une simple variable JavaScript (`pageAnnonce`). Elle repart à 1 dès qu'on filtre, cherche ou recharge la page.

Est-ce que le code est « débutant » ?

Oui, entièrement. Il n'y a **aucune librairie**, aucune abstraction compliquée : - que des variables, un `if`, une boucle `forEach` ; - une seule opération « maligne » : `.slice(debut, debut + parPage)`, qui est une fonction de base de JavaScript ; - les boutons sont du HTML généré dans une chaîne de caractères (comme partout ailleurs dans le projet).

Un correcteur peut demander de l'expliquer ligne par ligne sans piège : on charge, on découpe, on affiche, on change de tranche au clic.